Behind a Proxy

In many situations, you would use a **proxy** like Traefik or Nginx in front of your FastAPI app.

These proxies could handle HTTPS certificates and other things.

Proxy Forwarded Headers

A **proxy** in front of your application would normally set some headers on the fly before sending the requests to your **server** to let the server know that the request was **forwarded** by the proxy, letting it know the original (public) URL, including the domain, that it is using HTTPS, etc.

The **server** program (for example **Uvicorn** via **FastAPI CLI**) is capable of interpreting these headers, and then passing that information to your application.

But for security, as the server doesn't know it is behind a trusted proxy, it won't interpret those headers.

Technical Details

The proxy headers are:

- X-Forwarded-For
- X-Forwarded-Proto
- X-Forwarded-Host

Enable Proxy Forwarded Headers

You can start FastAPI CLI with the *CLI Option* `--forwarded-allow-ips` and pass the IP addresses that should be trusted to read those forwarded headers.

If you set it to `--forwarded-allow-ips="*"` it would trust all the incoming IPs.

If your **server** is behind a trusted **proxy** and only the proxy talks to it, this would make it accept whatever is the IP of that **proxy**.

```bash
fast →
$ fastapi run
```

Ask AI

Redirects with HTTPS

For example, let's say you define a *path operation* `/items/`:

Python 3.10+

```python
from fastapi import FastAPI

app = FastAPI()


@app.get("/items/")
def read_items():
    return ["plumbus", "portal gun"]
```

If the client tries to go to `/items`, by default, it would be redirected to `/items/`.

But before setting the *CLI Option* `--forwarded-allow-ips` it could redirect to `http://localhost:8000/items/`.

But maybe your application is hosted at `https://mysuperapp.com`, and the redirection should be to `https://mysuperapp.com/items/`.

By setting `--proxy-headers` now FastAPI would be able to redirect to the right location. 😎

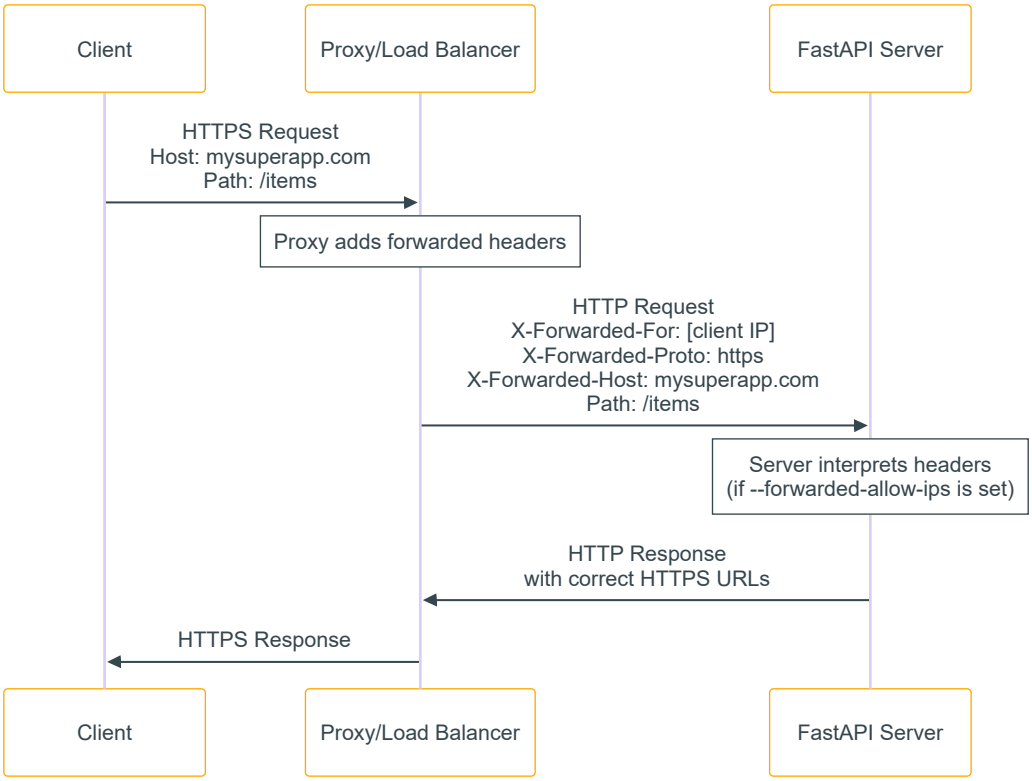
```
https://mysuperapp.com/items/
```

Tip

If you want to learn more about HTTPS, check the guide About HTTPS.

How Proxy Forwarded Headers Work

Here's a visual representation of how the **proxy** adds forwarded headers between the client and the **application server**:



The **proxy** intercepts the original client request and adds the special *forwarded* headers (`X-Forwarded-*`) before passing the request to the **application server**.

These headers preserve information about the original request that would otherwise be lost:

- **X-Forwarded-For**: The original client's IP address
- **X-Forwarded-Proto**: The original protocol (`https`)
- **X-Forwarded-Host**: The original host (`mysuperapp.com`)

When **FastAPI CLI** is configured with `--forwarded-allow-ips`, it trusts these headers and uses them, for example to generate the correct URLs in redirects.

Proxy with a stripped path prefix

You could have a proxy that adds a path prefix to your application.

In these cases you can use `root_path` to configure your application.

The `root_path` is a mechanism provided by the ASGI specification (that FastAPI is built on, through Starlette).

The `root_path` is used to handle these specific cases.

And it's also used internally when mounting sub-applications.

Having a proxy with a stripped path prefix, in this case, means that you could declare a path at `/app` in your code, but then, you add a layer on top (the proxy) that would put your **FastAPI** application under a path like `/api/v1`.

In this case, the original path `/app` would actually be served at `/api/v1/app`.

Even though all your code is written assuming there's just `/app`.

Python 3.10+

```python
from fastapi import FastAPI, Request

app = FastAPI()


@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

And the proxy would be **"stripping"** the **path prefix** on the fly before transmitting the request to the app server (probably Uvicorn via FastAPI CLI), keeping your application convinced that it is being served at `/app`, so that you don't have to update all your code to include the prefix `/api/v1`.

Up to here, everything would work as normally.

But then, when you open the integrated docs UI (the frontend), it would expect to get the OpenAPI schema at `/openapi.json`, instead of `/api/v1/openapi.json`.

So, the frontend (that runs in the browser) would try to reach `/openapi.json` and wouldn't be able to get the OpenAPI schema.

Because we have a proxy with a path prefix of `/api/v1` for our app, the frontend needs to fetch the OpenAPI schema at `/api/v1/openapi.json`.

Tip

The IP `0.0.0.0` is commonly used to mean that the program listens on all the IPs available in that machine/server.

The docs UI would also need the OpenAPI schema to declare that this API `server` is located at `/api/v1` (behind the proxy). For example:

```
{
    "openapi": "3.1.0",
    // More stuff here
    "servers": [
        {
            "url": "/api/v1"
        }
    ],
    "paths": {
            // More stuff here
    }
}
```

In this example, the "Proxy" could be something like **Traefik**. And the server would be something like FastAPI CLI with **Uvicorn**, running your FastAPI application.

Providing the `root_path`

To achieve this, you can use the command line option `--root-path` like:

```bash
```

If you use Hypercorn, it also has the option `--root-path`.

Technical Details

The ASGI specification defines a `root_path` for this use case.

And the `--root-path` command line option provides that `root_path`.

Checking the current `root_path`

You can get the current `root_path` used by your application for each request, it is part of the `scope` dictionary (that's part of the ASGI spec).

Here we are including it in the message just for demonstration purposes.

Python 3.10+

```python
from fastapi import FastAPI, Request

app = FastAPI()


@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

Then, if you start Uvicorn with:

```bash
```

The response would be something like:

```
{
    "message": "Hello World",
    "root_path": "/api/v1"
}
```

Setting the `root_path` in the FastAPI app

Alternatively, if you don't have a way to provide a command line option like `--root-path` or equivalent, you can set the `root_path` parameter when creating your FastAPI app:

Python 3.10+

```python
from fastapi import FastAPI, Request

app = FastAPI(root_path="/api/v1")


@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

Passing the `root_path` to `FastAPI` would be the equivalent of passing the `--root-path` command line option to Uvicorn or Hypercorn.

About `root_path`

Keep in mind that the server (Uvicorn) won't use that `root_path` for anything else than passing it to the app.

But if you go with your browser to http://127.0.0.1:8000/app you will see the normal response:

```
{
    "message": "Hello World",
    "root_path": "/api/v1"
}
```

So, it won't expect to be accessed at `http://127.0.0.1:8000/api/v1/app`.

Uvicorn will expect the proxy to access Uvicorn at `http://127.0.0.1:8000/app`, and then it would be the proxy's responsibility to add the extra `/api/v1` prefix on top.

About proxies with a stripped path prefix

Keep in mind that a proxy with stripped path prefix is only one of the ways to configure it.

Probably in many cases the default will be that the proxy doesn't have a stripped path prefix.

In a case like that (without a stripped path prefix), the proxy would listen on something like `https://myawesomeapp.com`, and then if the browser goes to `https://myawesomeapp.com/api/v1/app` and your server (e.g. Uvicorn) listens on `http://127.0.0.1:8000` the proxy (without a stripped path prefix) would access Uvicorn at the same path: `http://127.0.0.1:8000/api/v1/app`.

Testing locally with Traefik

You can easily run the experiment locally with a stripped path prefix using Traefik.

Download Traefik, it's a single binary, you can extract the compressed file and run it directly from the terminal.

Then create a file `traefik.toml` with:

```
[entryPoints]
  [entryPoints.http]
    address = ":9999"

[providers]
  [providers.file]
    filename = "routes.toml"
```

This tells Traefik to listen on port `9999` and to use another file `routes.toml`.

> **Tip**
>
> We are using port `9999` instead of the standard HTTP port `80` so that you don't have to run it with admin (`sudo`) privileges.

Now create that other file `routes.toml`:

```
[http]
  [http.middlewares]

    [http.middlewares.api-stripprefix.stripPrefix]
      prefixes = ["/api/v1"]

  [http.routers]

    [http.routers.app-http]
      entryPoints = ["http"]
      service = "app"
      rule = "PathPrefix(`/api/v1`)"
      middlewares = ["api-stripprefix"]
```

```
  [http.services]

    [http.services.app]
      [http.services.app.loadBalancer]
        [[http.services.app.loadBalancer.servers]]
          url = "http://127.0.0.1:8000"
```

This file configures Traefik to use the path prefix `/api/v1`.

And then Traefik will redirect its requests to your Uvicorn running on `http://127.0.0.1:8000`.

Now start Traefik:

```bash
```

And now start your app, using the `--root-path` option:

```bash
```

Check the responses

Now, if you go to the URL with the port for Uvicorn: http://127.0.0.1:8000/app, you will see the normal response:

```
{
    "message": "Hello World",
    "root_path": "/api/v1"
}
```

> **Tip**
>
> Notice that even though you are accessing it at `http://127.0.0.1:8000/app` it shows the `root_path` of `/api/v1`, taken from the option `--root-path`.

And now open the URL with the port for Traefik, including the path prefix:
http://127.0.0.1:9999/api/v1/app.

We get the same response:

```
{
    "message": "Hello World",
    "root_path": "/api/v1"
}
```

but this time at the URL with the prefix path provided by the proxy: `/api/v1`.

Of course, the idea here is that everyone would access the app through the proxy, so the version with the path prefix `/api/v1` is the "correct" one.

And the version without the path prefix (`http://127.0.0.1:8000/app`), provided by Uvicorn directly, would be exclusively for the *proxy* (Traefik) to access it.

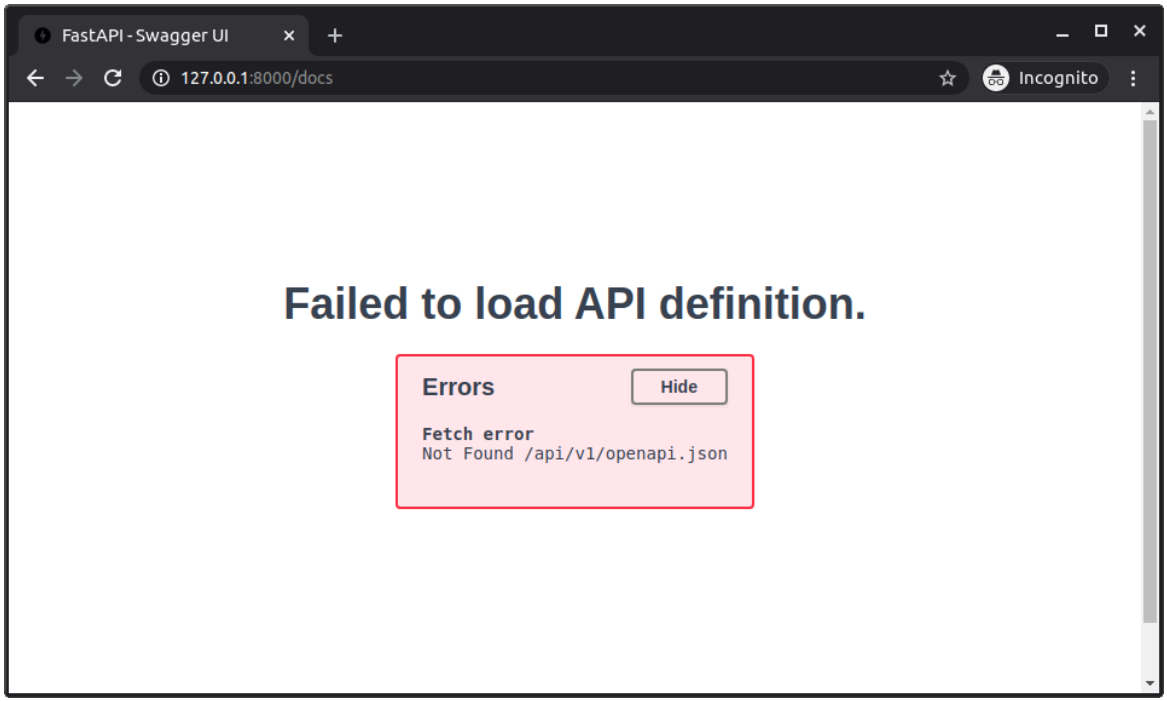
That demonstrates how the Proxy (Traefik) uses the path prefix and how the server (Uvicorn) uses the `root_path` from the option `--root-path`.

Check the docs UI

But here's the fun part. ✨

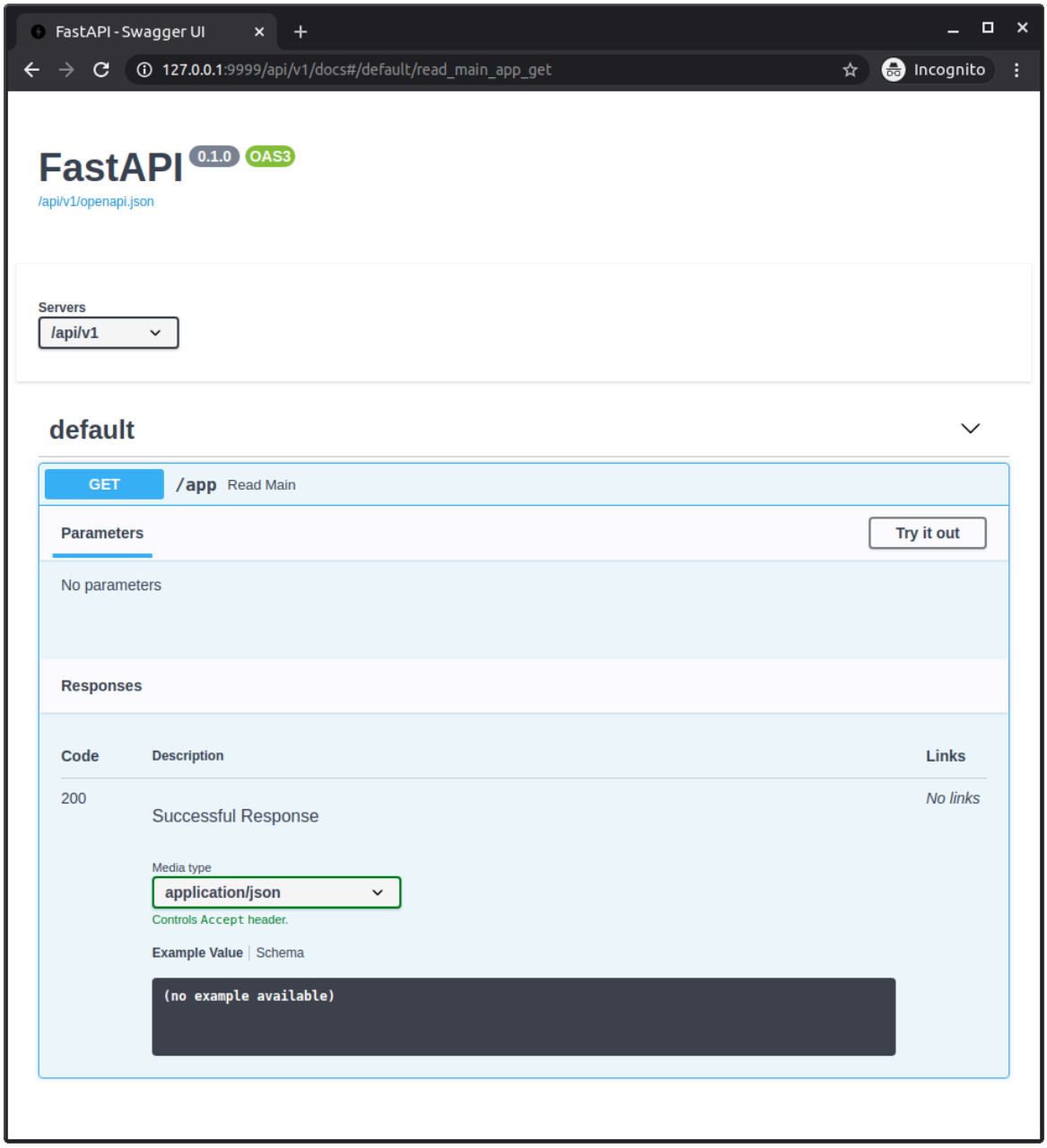
The "official" way to access the app would be through the proxy with the path prefix that we defined. So, as we would expect, if you try the docs UI served by Uvicorn directly, without the path prefix in the URL, it won't work, because it expects to be accessed through the proxy.

You can check it at http://127.0.0.1:8000/docs:



But if we access the docs UI at the "official" URL using the proxy with port `9999`, at `/api/v1/docs`, it works correctly! 🎉

You can check it at http://127.0.0.1:9999/api/v1/docs:



Right as we wanted it. ✔️

This is because FastAPI uses this `root_path` to create the default `server` in OpenAPI with the URL provided by `root_path`.

Additional servers

> **Warning**
>
> This is a more advanced use case. Feel free to skip it.

By default, **FastAPI** will create a `server` in the OpenAPI schema with the URL for the `root_path`.

But you can also provide other alternative `servers`, for example if you want *the same* docs UI to interact with both a staging and a production environment.

If you pass a custom list of `servers` and there's a `root_path` (because your API lives behind a proxy), **FastAPI** will insert a "server" with this `root_path` at the beginning of the list.

For example:

Python 3.10+

```python
from fastapi import FastAPI, Request

app = FastAPI(
    servers=[
        {"url": "https://stag.example.com", "description": "Staging environment"},
        {"url": "https://prod.example.com", "description": "Production environment"},
    ],
    root_path="/api/v1",
)


@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

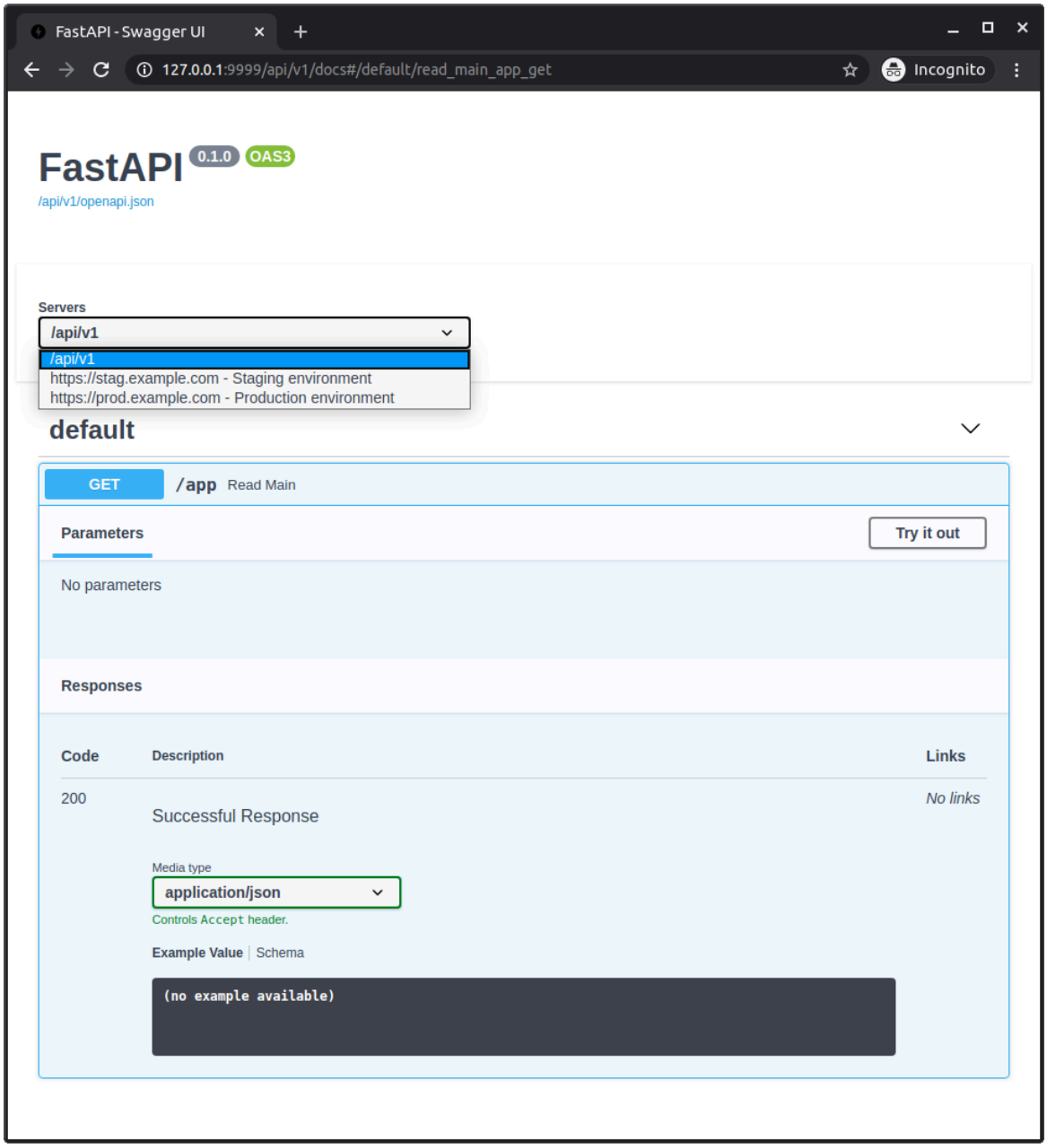
Will generate an OpenAPI schema like:

```jsonc
{
    "openapi": "3.1.0",
    // More stuff here
    "servers": [
        {
            "url": "/api/v1"
        },
        {
            "url": "https://stag.example.com",
            "description": "Staging environment"
        },
        {
            "url": "https://prod.example.com",
            "description": "Production environment"
        }
    ],
    "paths": {
            // More stuff here
    }
}
```

Tip

Notice the auto-generated server with a `url` value of `/api/v1`, taken from the `root_path`.

In the docs UI at http://127.0.0.1:9999/api/v1/docs it would look like:



Tip

The docs UI will interact with the server that you select.

Technical Details

The `servers` property in the OpenAPI specification is optional.

If you don't specify the `servers` parameter and `root_path` is equal to `/`, the `servers` property in the generated OpenAPI schema will be omitted entirely by default, which is the equivalent of a single server with a `url` value of `/`.

Disable automatic server from `root_path`

If you don't want **FastAPI** to include an automatic server using the `root_path`, you can use the parameter `root_path_in_servers=False`:

Python 3.10+

```python
from fastapi import FastAPI, Request

app = FastAPI(
    servers=[
        {"url": "https://stag.example.com", "description": "Staging environment"},
        {"url": "https://prod.example.com", "description": "Production environment"},
    ],
    root_path="/api/v1",
    root_path_in_servers=False,
)


@app.get("/app")
def read_main(request: Request):
    return {"message": "Hello World", "root_path": request.scope.get("root_path")}
```

and then it won't include it in the OpenAPI schema.

Mounting a sub-application

If you need to mount a sub-application (as described in Sub Applications - Mounts) while also using a proxy with `root_path`, you can do it normally, as you would expect.

FastAPI will internally use the `root_path` smartly, so it will just work. ✨